

Phase 1 — Measure properly, and learn what normal is

What Phase 1 is, what is finished, what is left, and what is wrong. Every number here is read from the live database, from git, or from the code at the moment this was built — 26 September 2026, 12:40.

Phase 1 is complete on both sides and merged. The platform measures 46 things about every capture instead of 10, judges whether each capture can be trusted, and has learned what normal looks like for this pump from 161 real captures rather than from one file somebody nominated. Laxman's eight tickets are merged, and the background worker left over from Phase 0 is built.

Three defects came in with that merge and are fixed. All three worked on the machine they were written on and silently did nothing elsewhere — a migration that could never run on an existing database, a test suite that skipped every database test without saying so, and a guard that let an empty catalogue through. They have a section of their own below.

1,601 automated tests across the four suites (862 platform, 364 chatbot, 353 frontend, 22 gateway), up from 58 at the end of Phase 0 — and all four were run while building this document: all passed. 25 commits, 218 files changed, 26,813 lines added and 2,056 removed.

The one thing Phase 1 cannot fix in software. The pump's vibration is so small next to the range the hardware is set to that two channels resolve it in about one and a half steps of the converter. Every shape measurement on those channels is describing rounding, not the machine. This needs somebody at the PLC, and it is the single highest-value open item in this document.

What Phase 1 was for, in one paragraph

Phase 0 made the numbers trustworthy. Phase 1 makes there be enough of them, and gives the system something to compare them against. Three questions, in order: **what can we measure about a capture** (it was 10 things, it is now 46), **can this capture be trusted at all** (eight checks that grade it), and **what does normal look like for this machine** (learned from its own history, not from a single saved file). Nothing in Phase 2 or Phase 3 — scoring how unusual a reading is, naming a fault — means anything without all three.

A fourth thing runs through all of it and is worth stating separately, because it is the difference between a demo and a system somebody can act on: **the platform must be able to say "I do not know"**. A machine with no baseline is not a healthy machine. A capture that could not be assessed did not pass. A feature with no learned normal is not normal. Each of those is a real distinction that a simpler design collapses into a reassuring green tick.

What is included in Phase 1

Eighteen tickets. Ten are Atharva's and all ten are done. Eight are Laxman's, and his branch is merged. Their detail lives in the 85-ticket sheet, which is not in this repository, so they are listed here by number and owner rather than described from memory — and what his code actually does is set out under the table.

Ticket	What it makes the system do	Status
VIK-018	13 more time-domain measurements Peak-to-peak, skewness, impulse and shape factors, burst count, shock index and others.	Done

Ticket	What it makes the system do	Status
VIK-019	13 more frequency-domain measurements Harmonic count, sideband spacing, spectral spread, broadband noise. One spectrum per segment, reused.	Done
VIK-020	Envelope measurements, including bearing bands Reads energy at the exact bearing fault frequencies from the catalogue. The roadmap calls this the most important group for the demo.	Done
VIK-021	Register the new measurements in the database One source of truth for all 46, so the code and the database cannot disagree about what a feature is.	Done
VIK-022	Judge whether a capture can be trusted Eight checks returning High / Medium / Low / Invalid, plus what failed. Every engine after this lowers its own confidence when trust is low.	Done
VIK-024	A table to hold learned normal Median and spread per sensor, channel and feature.	Done
VIK-025	Learn normal from history instead of one saved file Median and MAD, not average and standard deviation, so one bad capture cannot move the baseline. Refuses below a minimum sample count.	Done
VIK-026	Baseline lifecycle: start, roll, freeze, reset A reset starts a new version rather than editing the old one, so a finding can always be traced to the baseline that produced it.	Done
VIK-027	Report how healthy the baseline itself is Available or not, confidence, freshness, how many samples. Seven API endpoints; this is what unblocks Laxman's VIK-033.	Done
VIK-035	Put the shared maths in one package both services use Bearing maths, ISO tables, unit conversion, fault rules. Stops the platform and the chatbot reaching two different answers for the same bearing.	Done

Laxman's eight, merged

VIK-023, VIK-028, VIK-029, VIK-030, VIK-031, VIK-032, VIK-033 and VIK-034. His branch is merged and his commit says Phase 1 is complete. **This document does not tick them off individually, because it cannot check them:** the ticket sheet is not in the repository and his commits name only VIK-033. What follows is what the merged code demonstrably does, which is a different and more useful claim.

What landed	Ticket	Notes
The background worker	VIK-013	Uploads now parse, store and queue; a separate worker process runs plots, features and alerts. This was the Phase 0 leftover the last report flagged as starting to matter in Phase 2. Verified running against the merged schema.
Baseline health on screen	VIK-033	Reads the seven endpoints VIK-027 built. It is written not to fall back to looking sound when the check fails, which is the right call.
Capture trust on screen	—	An API layer over the VIK-022 engine that keeps the difference between a check that failed and one that could not run.
Thresholds scoped per sensor and equipment	—	Migration 029, plus a coverage matrix in the settings screen.

What landed	Ticket	Notes
3D equipment models	—	Nine machine types with a contract test for each.

What the system can do now that it could not

	End of Phase 0	Now
Measurements per capture	10	46
Bearing fault frequencies read	no	yes, from 88,734 catalogue entries
Capture trusted or not	no such idea	8 checks, 4 grades
What normal means	one file somebody nominated	learned from 161 captures
Can it say "I do not know"	no	yes, and it does
Baseline traceable to a finding	no	versioned, never edited
Shared maths	two copies	one package
Automated tests	58	1,601

What was built, in more detail

1. Forty-six measurements instead of ten (VIK-018, 019, 020, 021)

Ten measurements can tell you a machine is shaking harder than usual. They cannot tell you why. The extra thirty-six are the ones that distinguish an unbalanced rotor from a worn bearing from a loose foundation — things like how sharply the signal spikes, how energy is spread across frequencies, and how much sits at the exact frequencies a defect in this specific bearing would produce.

All 46 are registered in one place, so the code and the database cannot disagree about what a measurement is or what counts as normal for it. Eight of them are marked informational: they describe the signal rather than the machine's health, and a threshold on them would produce alarms that mean nothing.

2. Eight checks on whether a capture can be trusted (VIK-022)

Missing data, clipping, saturation, noise floor, bias drift, odd DC offset, unstable speed and a loose sensor. Each returns a grade and, when it fails, a sentence saying what is wrong in plain words. The thresholds were set by measuring this gateway rather than by choosing round numbers, which matters: the first version of the steadiness check called half the channels unstable, and the number it was reacting to was the length of the recording, not the machine.

A check that cannot run reports "not assessed", never "passed". Those are different facts and merging them is how a capture nobody could grade comes to look like a good one.

3. A learned normal, and the honesty around it (VIK-024, 025, 026, 027)

Before this, "healthy baseline" meant one capture somebody clicked a button on — and on this platform it was a copy of the very file being compared against it. The system now works out normal from the machine's own history, using the median and a robust spread so that one bad capture in twenty moves the answer by almost nothing.

It refuses rather than producing a weak baseline. A normal built from four captures is worse than no normal, because the system then trusts it.

Freeze is the part worth understanding. A baseline that keeps learning from a machine that is slowly wearing out will follow it down. The captures get worse gradually, normal slides with them, and the fault never looks abnormal because the yardstick moved. Freezing pins normal to a period somebody is willing to vouch for. It is the only way a slow decline shows up as one.

The baseline in force right now

Version and state	v5, active
Coverage	348 of 368 channel-feature pairs (95%)
Built from	124 captures at the median
Confidence	median 1.00; 22 feature(s) at or below 0.5
Freshness	window closed 4.8 days ago; 37 captures since

Read through the same read model the API serves, so this table is a caller of VIK-027 rather than a second opinion about the same rows.

4. One copy of the shared maths (VIK-035)

The platform and the chatbot both answer questions about the same bearing on the same machine, and until now they did it from separate code. If they disagreed, nothing would crash: one would report a bearing fault and the other would report nothing, months apart, and whichever a person happened to read would be the answer they acted on. Bearing formulas, ISO tables, unit conversion and fault rules now exist once, in `backend/vibcore/`, imported by both.

What was found wrong and fixed along the way

None of these was on the ticket list. All of them were making the system's answers wrong while looking entirely normal on screen, which is the only kind of bug that matters at this stage.

What was wrong	Why it mattered
Measurements were reading the sensor's resting position, not the vibration	Eight of thirteen time measurements were computed without removing each channel's standing bias. On one channel that bias was larger than the vibration itself, so the system reported every single sample as an impact. 1,163 of 1,479 critical readings were false; five of eight channels read critical while the pump was running normally.
The shaft speed was wrong on every channel	The speed was taken from the tallest line in the spectrum, which on this pump is two or four times the true speed. A speed wrong by two puts every bearing frequency out by two. Now resolved from the machine nameplate, cross-checked, and marked unusable when it cannot be established.
Whole windows of samples were being thrown away	The gateway crashed on a timestamp the PLC had not set yet, and the error escaped in a way that silently dropped the whole message. Five complete recordings were lost before it was found.
Saving the settings screen erased the sensor sensitivities	The settings form had no field for per-channel sensitivity, and saving replaced the whole stored map with what the form knew about. Opening the page and pressing save — changing nothing — wiped the figures the gateway had supplied.
A baseline could be built across a settings change	Fourteen of forty-two measurements move by more than a quarter when the recording length or sample rate changes. One measurement read seven standard deviations out, entirely because somebody changed a setting. A baseline is now tied to the recording shape it was learned from.

What was wrong	Why it mattered
Nine fake captures were being counted as evidence about a real pump	One synthetic test file, uploaded nine times, sitting in the history looking like nine independent observations. The median ignored them, as it should; the 95th percentile could not, and reported 0.29962 against a real machine level of 0.01175 — twenty-five times too high, presented as an ordinary upper limit for a healthy pump. Removed, and backed up first.
The converter step was taken on trust from a config file	The trust checks divide the vibration by the converter's step size, and that came from a number nobody had verified — while two config files in this deployment disagree about it. The step is now measured from the samples themselves, and the system says so when the measurement and the record disagree.

How these were found. Mostly by mutation testing: deliberately breaking one guard in the code and checking that a test notices. A test that still passes when the thing it guards is broken is not a test. Applied to every module written in Phase 1 — and the first time it was run on the frequency measurements, 11 of 18 deliberate breakages went unnoticed, which is how the sideband bug was found. It is also how a gap in the baseline work was found this week: nothing covered a baseline that was built but never switched on.

What the merge broke, and what fixed it

Three defects came in with the merge. All three share a shape worth naming, because it is the one that survives review: **they work on the machine they were written on and silently do nothing everywhere else**. None of them failed loudly.

What was wrong	What it means
A migration that can never run on an existing database	processing_jobs was added as revision 021, ahead of the migration already holding that number. Alembic records only where a database is now, never the path it took, so a database at 028 computes 028 to 029 and never walks back to pick up something inserted at 021. The table is never created, alembic still reports head, and the first symptom is the worker dying. Fresh databases were fine, which is where it was written. Moved to 030, the end of the chain, which is the only place every database is guaranteed to pass through.
Every database test was skipping	The new test setup reads the database address from the environment, but this repository keeps it in backend/.env, and it expected the test database to have been created by hand. Neither was true here, so all of it skipped — including the twenty-six covering the baseline lifecycle. A skip is not a failure and reads exactly like a pass. It now falls back to the .env file and creates the database itself.
Five bearing tests failed instead of skipping	Their guard asked whether the catalogue table could be queried, which an empty table answers yes to. Pointed at a fresh test database it waved through zero rows. The guard now requires rows, and copies the 88,734 from the development database when it has them.

The guard that now exists, and the one that does not work. The obvious test for the migration bug — migrate a second database to an older revision, then to head, and compare — does not catch it. It was written first, the bug was recreated behind it, and it passed: under the new chain, reaching 028 necessarily passes through anything inserted before 028. The chain is internally valid; what is wrong is its relationship to databases that already exist, and the files hold no record of that.

alembic/released_revisions.txt is that record — one line per released revision and the parent it shipped with. A released revision may not be renumbered or re-parented, and a new one must hang off the last released revision. Recreating the original mistake now fails with *"revision 022 was released with parent '021' and now claims '021a'"*.

What still needs fixing

1. The hardware range is wrong for this machine — needs somebody at the PLC

This is the most valuable open item and it is not a software problem.

The sensors are set to 100 mV/g, which lets the converter cover plus or minus 50 g. This pump uses about a thousandth of that. Measured across the twenty most recent captures, here is how many steps of the converter the actual vibration spans on each channel — below about three, the channel is describing the rounding rather than the machine:

Channel	Steps	Reading
ch0	2.4	reporting rounding
ch1	2.5	reporting rounding
ch2	9.4	coarse
ch3	5.5	coarse
ch4	3.1	coarse
ch5	1.5	reporting rounding
ch6	1.5	reporting rounding
ch7	7.5	coarse

The gateway's own configuration file asks for 500 mV/g on the first two channels, which would narrow the range to plus or minus 10 g and give five times the resolution. The samples show the PLC applied 100 to all eight. Somebody has already worked out what the fix is; the PLC is not honouring it. **What is needed:** someone with access to the PLC configuration to confirm what sensitivity it is actually applying, and whether the transducers on channels 0 and 1 are 500 mV/g parts.

Until then 776 of 1,288 channel assessments fail the noise-floor check, and those failures are real.

2. No data has arrived since 22 September, 04:53

The gateway has been silent. Everything in this document is built from the 161 captures already stored, covering 17 September to 22 September. Nothing is wrong with the platform; the source stopped publishing.

3. Two thresholds cannot be calibrated yet

The steadiness check needs about ten shaft revolutions in each half of a recording to be meaningful. At the current recording length this pump gives 9.87 — just under. The threshold is set at 10 and the check stands down rather than guessing, which is correct but means it is not doing any work. A slightly longer recording fixes

it, and the same change would also make one more bearing frequency readable.

4. Twenty-two features have untrustworthy percentiles

These were checked individually and are genuine, not contamination: measurements like the dominant frequency legitimately jump between separate values rather than varying smoothly around one. Their median and spread are usable; their 5th and 95th percentiles describe a different population and should not be read as limits. The system now marks them so nothing downstream reads them as limits by accident.

5. Open from Phase 0 — now closed

VIK-013, the background worker, was the outstanding Phase 0 item in the last report. It is now built and merged: uploads parse, store and queue, and a separate worker process runs the rest. It was verified against the merged schema once its migration was moved to the end of the chain — before that it could not start on this database at all.

The state of the data today

What	Count	Note
Captures stored	161	all from the real gateway, 50 kSPS for 0.278 s
Uploads	162	the nine synthetic ones removed
Measurements computed	45,632	46 per channel per capture
Trust assessments	1,449	512 high, 421 medium, 355 low
Learned baselines	1,760	across 5 versions, one in force
Bearings in the catalogue	88,734	loaded in Phase 0

Eight database migrations were added in Phase 1, numbered 021 to 028. They add the measurement register, the envelope definitions, the trust assessments, the learned-normal table, the recording-shape scoping and the baseline lifecycle. Migration 028 also adopted the five baselines that already existed, timestamped from when each was actually built rather than from the day the migration ran.

How to check any of this yourself

Nothing in this document has to be taken on trust. Each of these can be run now:

To check	Run
Run every test	cd backend && .venv\Scripts\python -m pytest tests/ -q
Rebuild this document from live data	cd backend && .venv\Scripts\python scripts/build_phase1_report.py
Ask the platform how good its baseline is	GET /api/v1/learned-baselines/health?sensor_id=...
See every baseline version and why each was started	GET /api/v1/learned-baselines/versions?sensor_id=...
Re-grade every stored capture	cd backend && .venv\Scripts\python scripts/reassess_quality.py
Restore the nine deleted synthetic uploads	cd backups/synthetic_uploads && python restore.py --dry-run

What happens next

Atharva: Phase 1 is finished. Phase 2 is next — scoring how unusual each reading is, against the right kind of normal. That work reads the baselines and the trust grades built here, which is why it could not start earlier.

Laxman: his branch is merged and green. Two things worth carrying forward. A new migration goes at the end of the chain and gets a line in `released_revisions.txt`; a test now enforces both. And a test that skips is not a test that passes — the suite was skipping every database test on this machine without anything saying so. Worth running with `-rs` occasionally to see what is being skipped.

Whoever owns the hardware: the sensitivity question above. It costs one conversation and it is currently limiting what every measurement on five of eight channels can say.

Built from the repository at commit 904df77, the live database, and the test suites as they ran at 26 September 2026, 12:40. The script that produced it is `backend/scripts/build_phase1_report.py`.